



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 01 — Python básico

Bloque 02

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



La idea en una frase

“

En Python todo valor es un **objeto** con un tipo que el lenguaje **deduce solo** al asignarlo —sin declararlo como en C o Java—, y los primitivos con los que trabajaremos son `int`, `float`, `bool`, `str` y `None`.



Números enteros (`int`)

Los enteros no tienen límite de tamaño: puedes tener números enormes sin preocuparte.

```
3      # => 3
-10    # => -10
1000000 # => 1000000
```

Operaciones aritméticas

Operador	Operación	Ejemplo	Resultado
<code>+</code> / <code>-</code>	Suma y Resta	<code>1 + 1</code> <code>8 - 1</code>	<code>2</code> <code>7</code>
<code>*</code>	Multiplicación	<code>10 * 2</code>	<code>20</code>
<code>()</code>	Precedencia	<code>(1 + 3) * 2</code>	<code>8</code> (los paréntesis fuerzan el orden)
<code>**</code>	Potencia	<code>2 ** 3</code>	<code>8</code> (2 elevado a 3)
<code>%</code>	Módulo (resto)	<code>10 % 3</code>	<code>1</code> (resto de 10/3: 3×3=9, resto 1)



División y decimales (float)

La división con `/` **siempre devuelve un float** (decimal), incluso entre enteros:

```
35 / 5 # => 7.0 (float, no int)
34 / 5 # => 6.8
```

Si quieres un resultado entero con truncado de decimales, usa `//`:

```
34 // 5 # => 6 (trunca los decimales)
35 // 5 # => 7
```

Cuando un operador es float, el resultado siempre es float:

```
3 * 2.0 # => 6.0
3.5 + 1.2 # => 4.7
```

⚠ Precisión IEEE 754: `0.1 + 0.2` no da 0.3 exacto por la representación interna en punto flotante. No es un error de Python. Para cálculos que requieren precisión exacta (dinero), se usa el módulo `decimal`.



Booleanos (`bool`) y comparaciones

Dos valores posibles: `True` y `False` (con mayúscula inicial obligatoria).

Operador	Acción	Ejemplo	Resultado
<code>not</code>	Invierte el valor	<code>not True</code> <code>not False</code>	<code>False</code> <code>True</code>
<code>=</code> / <code>≠</code>	Igualdad / Desigualdad	<code>1 = 1</code> <code>1 ≠ 1</code>	<code>True</code> <code>False</code>
<code><</code> / <code>></code>	Menor / Mayor que	<code>1 < 10</code> <code>1 > 10</code>	<code>True</code> <code>False</code>
<code>≤</code> / <code>≥</code>	Menor/Mayor o igual	<code>2 ≤ 2</code> <code>2 ≥ 2</code>	<code>True</code> <code>True</code>

Comparaciones encadenadas

```
1 < 2 < 3 # True (equivale a 1 < 2 and 2 < 3)
2 < 3 < 2 # False
```



`=` compara valores: `1 = 1.0` es `True`. `is` compara identidad: `1 is 1.0` es `False`.



Cadenas de texto (`str`) y f-strings

Se delimitan con comillas simples `'` o dobles `"`. Permiten índices positivos (inicio en 0) e índices negativos (-1 es el último):

```
texto = "Hola mundo"
texto[0]      # 'H' (primer carácter)
texto[-1]     # 'o' (último carácter)
"Hola " + "mundo!" # Concatenación con +
```

Formateo: `.format()` vs f-strings

Método	Ejemplo	Resultado
<code>.format()</code>	<code>"{0} sé rápido".format("Jack")</code>	<code>"Jack sé rápido"</code>
f-strings (3.6+)	<code>f"{nombre} come {comida}"</code>	<code>"Bob come Lasaña"</code>



Usa **f-strings** en código nuevo: son directos, legibles y recomendados desde Python 3.6.



Métodos útiles de str

```
"Hola Mundo".lower()      # "hola mundo"
"Hola Mundo".upper()      # "HOLA MUNDO"
"Hola Mundo".replace("Mundo", "Py") # "Hola Py"
"hola mundo".split()      # ["hola", "mundo"] (divide por espacios)
"  hola  ".strip()        # "hola" (elimina espacios al inicio y final)
```



El objeto `None`

`None` representa la ausencia de valor (tipo `NoneType`). No uses `=` para comparar con `None`. Usa `is`:

Regla de Comparación	Sintaxis correcta	Resultado
Usa siempre <code>is</code> (nunca <code>=</code>)	<code>"etc" is None</code>	<code>False</code>
	<code>None is None</code>	<code>True</code>



Valores falsy y conversiones de tipo

Se evalúan como `False` en contexto booleano: `None`, `0`, `""`, `[]`, `{}` y `set()`.

```
bool(0)      # => False
bool("")     # => False
bool([])     # => False
bool({})     # => False
bool(set())  # => False
```



Todo lo demás se evalúa como `True` (**Truthy**). Conocerlo simplifica condiciones y validaciones.

Funciones de conversión

```
int(3.7)      # 3: trunca, no redondea
float(5)      # 5.0
str(100)      # "100"
bool(1)       # True
bool(0)       # False
bool("hola")  # True
bool("")      # False
```



Mini-chequeo — Tipos de datos

1. ¿Qué devuelve `10 / 2` en Python 3? ¿Y `10 // 2`?
2. ¿Cuál es la diferencia entre `=` y `is`?
3. ¿Por qué `0.1 + 0.2` no da exactamente `0.3`?

Respuestas:

1. `10 / 2` devuelve `5.0` (float). `10 // 2` devuelve `5` (int, división entera).
2. `=` compara valores: `1 = 1.0` es True. `is` compara identidad de objeto: `1 is 1.0` es False.
3. Por la precisión del formato de punto flotante. 0.1 y 0.2 no se representan exactamente en binario, por lo que la suma acumula un pequeño error. No es un bug de Python.



Variables y convención PEP 8

Las variables almacenan datos y las colecciones agrupan varios datos: listas (mutables), tuplas (inmutables), diccionarios (clave:valor) y conjuntos (sin duplicados).

En Python no hace falta declarar variables: asignas un nombre a un valor con `=`:

```
nombre = "Ana"  
edad = 22  
activa = True
```

Convención	Uso	Ejemplo
snake_case	Variables y funciones (PEP 8)	mi_variable = 5
PascalCase	Clases	PersonaCliente
UPPER_CASE	Constantes	IVA_GENERAL
camelCase	Funciona, pero no es la guía oficial	miVariable = 5



Acceder a una variable que no existe produce `NameError`: `name 'otra_variable' is not defined`



Listas (colecciones mutables)

Las listas son la colección más usada en Python. Almacenan una secuencia de elementos y son **mutables** (puedes cambiarlas después de crearlas).

```
# Crear listas
lista = []
otra_lista = [4, 5, 6]
```

Añadir y eliminar elementos

```
lista.append(1) # [1]
lista.append(2) # [1, 2]
lista.append(4) # [1, 2, 4]
lista.append(3) # [1, 2, 4, 3]
lista.pop()     # ⇒ 3 (elimina y devuelve el último)
```

Acceso por índice

Los índices empiezan en 0. Python también permite índices negativos (-1 es el último):

```
lista = [1, 2, 4, 3]
lista[0]     # ⇒ 1
lista[-1]    # ⇒ 3
# lista[4]   # IndexError: list index out of range
```



Slicing (rebanado de listas y cadenas)

El *slicing* extrae una porción con la sintaxis `secuencia[inicio:final:paso]`. El índice final **no se incluye**.

```
lista = [1, 2, 4, 3]

lista[1:3] # => [2, 4]      (del índice 1 al 2, sin incluir el 3)
lista[2:]  # => [4, 3]      (desde el índice 2 hasta el final)
lista[:3]  # => [1, 2, 4]    (desde el inicio hasta el índice 2)
lista[::2] # => [1, 4]       (cada dos elementos)
lista[::-1] # => [3, 4, 2, 1] (invertir la lista completa)
```



El índice final indicado en la cota superior del slicing jamás se incluye en el resultado devuelto.



Otras operaciones con listas

```
del lista[2]           # Elimina el elemento en índice 2 (el 4): lista ahora es [1, 2, 3]

lista + otra_lista     # ⇒ [1, 2, 3, 4, 5, 6] (no modifica las originales)

lista.extend(otra_lista) # Añade otra_lista al final de lista (modifica lista)

1 in lista             # ⇒ True (comprobar existencia)

len(lista)             # ⇒ 6 (longitud)
```



Tuplas e inmutabilidad

Las tuplas son como las listas, pero **inmutables**: no puedes cambiar nada una vez creadas. Se definen con paréntesis `()`.

```
tupla = (1, 2, 3)
tupla[0]      # => 1
# tupla[0] = 3 # TypeError: 'tuple' object does not support item assignment
```

Operaciones válidas

```
len(tupla)      # => 3
tupla + (4, 5, 6) # => (1, 2, 3, 4, 5, 6)
tupla[:2]       # => (1, 2)
2 in tupla      # => True
```

i Usa una tupla cuando los valores no deben cambiar: coordenadas (x, y), configuraciones (ancho, alto), registros.



Desempaquetado e intercambio

Una de las cosas más elegantes de Python

Asignar múltiples variables de golpe, sin variable temporal:

```
a, b, c = (1, 2, 3)    # a=1, b=2, c=3
d, e, f = 4, 5, 6      # sin paréntesis (tupla automática)

# Intercambiar valores sin variable temporal
e, d = d, e            # d=5, e=4
```



El desempaquetado también funciona con listas y strings: `x, y = [10, 20]` o `a, b = "hi"`.



Diccionarios (clave:valor)

Los diccionarios almacenan pares `clave:valor` delimitados por llaves `{}`. Ideales para datos estructurados.

```
# Diccionario vacío
dicc_vacio = {}

# Diccionario prellenado
usuario = {"uno": 1, "dos": 2, "tres": 3}

# Acceso directo por clave (lanza KeyError si no existe)
usuario["uno"]          # => 1
# usuario["cuatro"]      # KeyError!

# Método seguro .get()
usuario.get("uno")       # => 1
usuario.get("cuatro")    # None (no lanza error)
usuario.get("cuatro", 4) # => 4 (valor por defecto)
```



Diccionarios: claves, valores y modificaciones

```
list(usuario.keys())    # => ["uno", "dos", "tres"]
list(usuario.values())  # => [1, 2, 3]

# Añadir, modificar y eliminar
usuario["cuatro"] = 4    # Añadir o modificar
usuario.setdefault("cinco", 5) # Solo si la clave no existe
del usuario["uno"]       # Eliminar una clave

# Comprobar existencia
"uno" in usuario         # => True
1 in usuario             # => False (1 es un valor, no una clave)
```



Acceder con `usuario["clave"]` lanza `KeyError` si la clave no existe. Usa `.get()` cuando pueda faltar.

Conjuntos (set)

Los conjuntos almacenan elementos **sin duplicados** y permiten operaciones de teoría de conjuntos.

```
# Conjunto vacío (¡cuidado! {} crea un dict, no un set)
conjunto_vacio = set()

# Conjunto con valores (los duplicados se eliminan solos)
un_conjunto = {1, 2, 2, 3, 4}  # ⇒ {1, 2, 3, 4}
otro_conjunto = {3, 4, 5, 6}

un_conjunto & otro_conjunto    # ⇒ {3, 4}  intersección
un_conjunto | otro_conjunto    # ⇒ {1, 2, 3, 4, 5, 6}  unión
{1, 2, 3, 4} - {2, 3, 5}      # ⇒ {1, 4}  diferencia

un_conjunto.add(5)            # ⇒ {1, 2, 3, 4, 5}
2 in un_conjunto              # ⇒ True
```



Resumen de colecciones

Colección	Mutable	Duplicados	Sintaxis / Acceso
Lista	Sí	Sí	<code>[1, 2, 3]</code> Por índice (<code>lista[0]</code>)
Tupla	No	Sí	<code>(1, 2, 3)</code> Por índice (<code>tupla[0]</code>)
Diccionario	Sí	Claves NO	<code>{"k": "v"}</code> Por clave (<code>dict["k"]</code> o <code>.get()</code>)
Set	Sí	NO	<code>{1, 2, 3}</code> Operaciones de conjuntos / <code>in</code>

`{}`

crea un diccionario vacío; para un set vacío escribe `set()` .



Mini-chequeo — Colecciones

1. ¿Cuál es la diferencia entre una lista y una tupla?
2. ¿Qué devuelve `{1, 2, 2, 3}` ?
3. ¿Cómo accedes al último elemento de una lista `mi_lista` ?

Respuestas:

1. La lista es mutable (puedes añadir, eliminar y cambiar elementos). La tupla es inmutable (una vez creada, no cambia).
2. `{1, 2, 3}` : los conjuntos eliminan duplicados automáticamente.
3. `mi_lista[-1]` : el índice -1 accede al último elemento.



Resumen — Bloque 02

- Los tipos primitivos son `int`, `float`, `bool`, `str` y `None`; Python deduce el tipo automáticamente.
- La división `/` devuelve float; `//` devuelve entero con truncado.
- Usa `is` para comparar con `None` y f-strings para formatear texto de forma moderna.
- Las variables no se declaran: simplemente asignas un valor con `=`.
- Las listas son mutables y usan índices; las tuplas son inmutables; los diccionarios usan claves; los conjuntos eliminan duplicados.
- El slicing `lista[inicio:final:pasos]` extrae porciones de listas y strings de forma elegante.



Vocabulario rápido

Término	Idea general
<code>int</code>	Entero, sin límite de tamaño
<code>float</code>	Decimal (punto flotante)
<code>bool</code>	Booleano: <code>True</code> o <code>False</code>
<code>str</code>	Cadena de texto
<code>None</code>	Representa "nada"
Tipado dinámico	El tipo se deduce al asignar
f-string	Formato moderno <code>f" ... "</code>
Concatenación	Unir strings con <code>+</code>

Término	Idea general
<code>snake_case</code>	Minúsculas con guion bajo
Mutable	Puede cambiar tras crearse
Immutable	No puede cambiar
Slicing	Extraer una porción
Desempaquetado	Asignar varias variables de golpe
Clave:valor	Par de datos en un diccionario
Set	Conjunto sin duplicados

